QLion Gold card guide

Document version 4.1

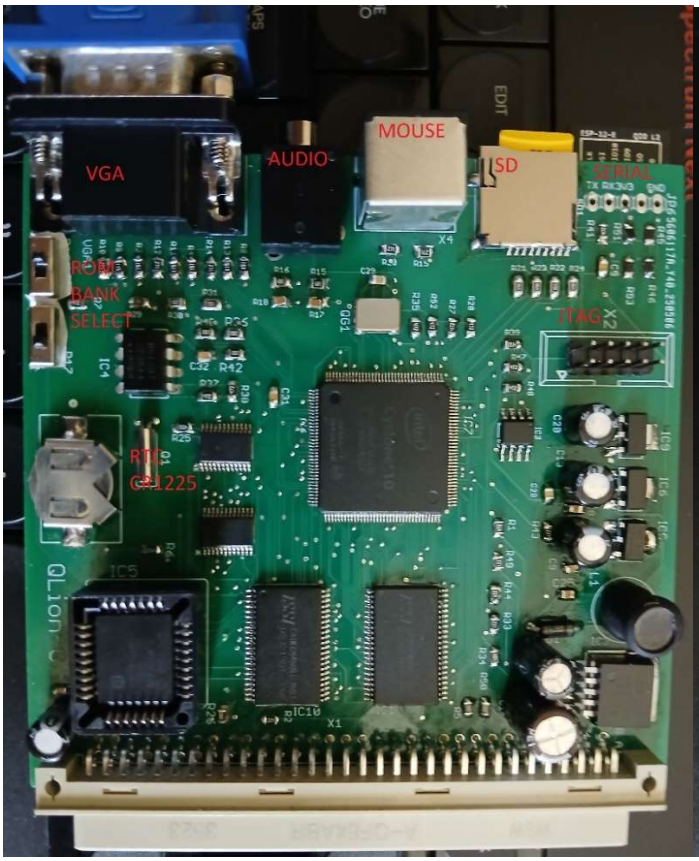
Table of Contents

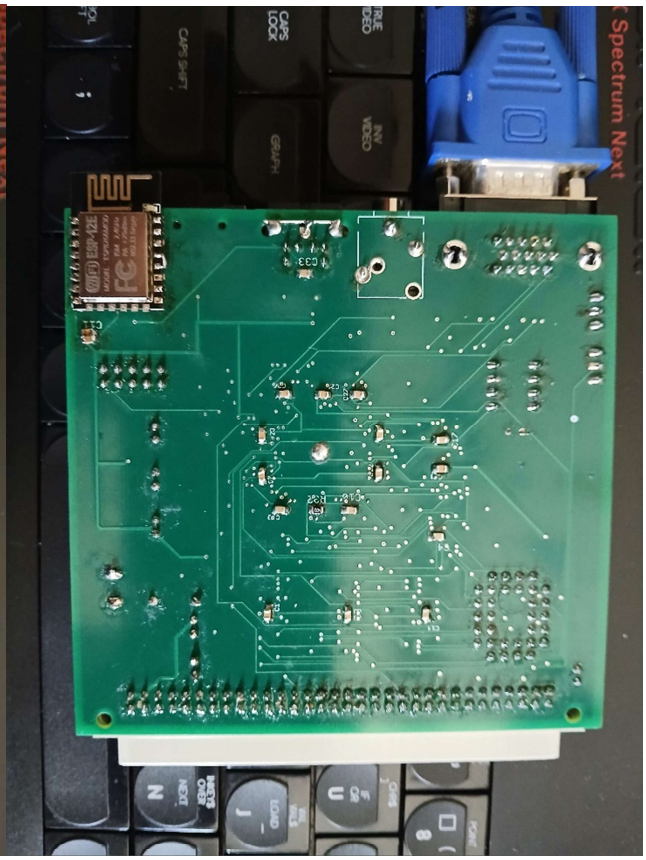
FEATURE OVERVIEW 2
INSTALLATION 3
INTRODUCTION DEMO PROGRAMS 3
FLASH ROM 4
NEW VIDEO MODES 4
RESOLUTION MIX 6
SPRITES 6
YM/AY AUDIO 9
PCM AUDIO 9
SERIAL/ESP 9
REAL TIME CLOCK 10
STORAGE QLSD 11
FAT SUPPORT UTILITIES 11
SUPERBASIC EXTENSIONS 11
MEMORY MAP 13
MEMORY MAPPED I/O REGISTERS 14
COPYRIGHTS 15

Feature overview

- TG68K cpu core, MC68000 or MC68020 compatible CPU switchable on the fly
- VGA output 1024x768 @60hz
- 4 Mbytes RAM + VRAM (3712 Kbytes RAM)
- 512K of flash rom provide 4 different rom sets to boot from (to replace one you need a flash programmer)
- Rom can be made writable and be patched while running
- QLSD storage (Lion SPI hardware Interface)
- AY/YM chip emulation (not QSound compatible but sound sbasic command same as QSound's)
- PCM Audio with 8K dedicated hardware buffer and a simple player of raw samples.
- Extended Mode 8 with 16 colours by disabling the 'FLASH' bit and special FINK and FPAPER commands to address it
- New video modes with 16, 256 and 4096 colours and resolutions up to 1024x768 (4 colours)
- Qimi mouse compatibility (via PS/2 port)
- 90* hardware sprites 16x16 15 colours. Sprite palette of 15 out of 256 colours for each sprite
- DS1307 battery backed-up RTC support via specialized commands
- Fast serial port SER3 and option for ESP12E onboard (alias ESP1)
- Ability to access the sd card FAT32 FS and transfer files via SDIR, SCOPY, QCOPY utilities
- SMSQ/E with support of aurora graphic modes 256 colours @ 512x384, 4 colours @1024x768.
- Fast counter, vsync, hsync, video scan line, card ID readable registers.
- Speed control with FSLOW command to enable playing standard QL games.

  * For version of firmware below 3.3 the card supported maximum of 50 sprites





Installation

The card contains some tall components so be careful while inserting it as with any other QL card.
A HC SD card (usually a card 8GB or bigger) freshly formatted with FAT32 is required, as with the other implementations of qlsd the QXL.WIN (to QXL8.WIN) image(s) must be in the first 16 fat table entries and be unfragmented so they better be the first files copied to the sd card after format.
A QXL.WIN with demo programs and drivers is provided with the card.
You'll need a CR1225 or CR1220 battery for the RTC, positive side up.
Don't use the card with another card in the expansion port or any device that uses the rom port.
Microdrive and microdrive emulators work with the card and the card's patched Minerva rom, you'll see a screen flicker while accessing them but this is normal.
QL Net works using the ND-QLi NET driver by Martyn Hill.
Making backup copies of the files of the SD card you use is strongly recommended.

Introduction demo programs

Demo programs are located in the demos directory.
A sprite demo game is the Galaxp, a galaxians(ish) clone. To play enter: EW demos_galaxp
The game starts with the enter key and exits with the esc key.
To see the use of sprites in basic enter: lrun demos_testspr

To play a ym file connect pc speakers to the audio port and enter: ew ymplay,#1;'demos_test_ym' (it takes few seconds to start playing because it buffers the file in ram) or use the same command with arka_ym and test2_ym files, To stop playback press the ESC key.
To play a small test digital audio sample enter ew dplay;'demos_pol_raw 8000' or lrun demos_testsample
To play a AY/YM sound in basic use the command: SOUND channel, frequency, volume (volume=0..15)
To stop the sound: SOUND channel or NOSOUND mutes all channels (channel = 0 to 2).

To view a couple of 256x256x256 colour pictures use: lrun demos_test256.
lrun demos_test256c demonstrates the pch256 command.
To view demo pictures in 512x256 512x384 modes : lrun demos_test512
To see hi colour picture in 256x256 mode and a hires 1024x768 pic use: lrun demos_testhi
To see the use of 16 colors in standard video mode enter: lrun demos_testcols

SMSQE

lrun tsms boots smsqe (wait a few seconds)
While in smsqe different modes can be selected using
lrun m384256 switches to 512x384 256 colors, after you may resize the windows to occupy full screen.
lrun m512 switches back to 512x256 4 colors
lrun m1024 switches to 1024x768 4 colors
lrun qle384 or **qle1024** runs the qle after swapping win1_ with win2_ (wait a few seconds) in 512x384 256 colors or 1024x768 4 colors. This assumes the QLE distribution exists at the WIN2_ device.

Flash Rom

Four sets of rom/drivers/utilities can be written in the flash IC. The active set is selected by the 2 switches on the pcb. The 4 sets are the 4 quarters of the 512K 29F040 flash ic so 128K each.
The first 96K of the selected set are used as QL's rom memory. A user can write using a flash programmer (e.g. TL866 and plcc32 adapter) the sets of roms and drivers-utilities they prefer.
Mind that a JS rom has to be patched for the bigger memory and the qlsd driver is needed for sd access.
A patched JS rom for 4MB is now included by default in the flash ROM.
Also for Microdrives to work the preprogrammed Minerva patched rom should be used.

New Video modes

The following video modes are supported by the card.

Video mode list

Mode number	Resolution/colours	Screen no.	Address	Pixel colour pattern
10	1024x768x4	Screen 1	$4C0000	Standard mode 4
17	512x256x16	Screen 1	$4C0000	IRGB
21	512x384x16	Screen 1	$4C0000	IRGB
32	256x256x256	Screen 1	$4C0000	RRRGGGBB
33	512x256x256	Screen 1	$4C0000	RRRGGGBB
37	512x384x256	Screen 1	$4C0000	RRRGGGBB
48	256x256x4096	Screen 1	$4C0000	RRRRXGGGGXBBBBXX
81	512x256x16	Screen 2	$4D0000	IRGB
96	256x256x256	Screen 2	$4D0000	RRRGGGBB
128	256x256x8	Screen 1	$20000	Standard mode 8
129	512x256x4	Screen 1	$20000	Standard mode 4
144	256x256x16	Screen 1	$20000	IGRB (standard QL pattern)
193	512x256x4	Screen 2	$28000	Minerva screen 2

You enable a video mode by writing a byte with the mode number to the mode register at address 8388622.

Pixel pattern notation: R=red, G=green, B=blue, I=higher intensity, X=not used

The way the screens are structured in video memory is similar to the standard mode as a raster scan from left to right and top to bottom.

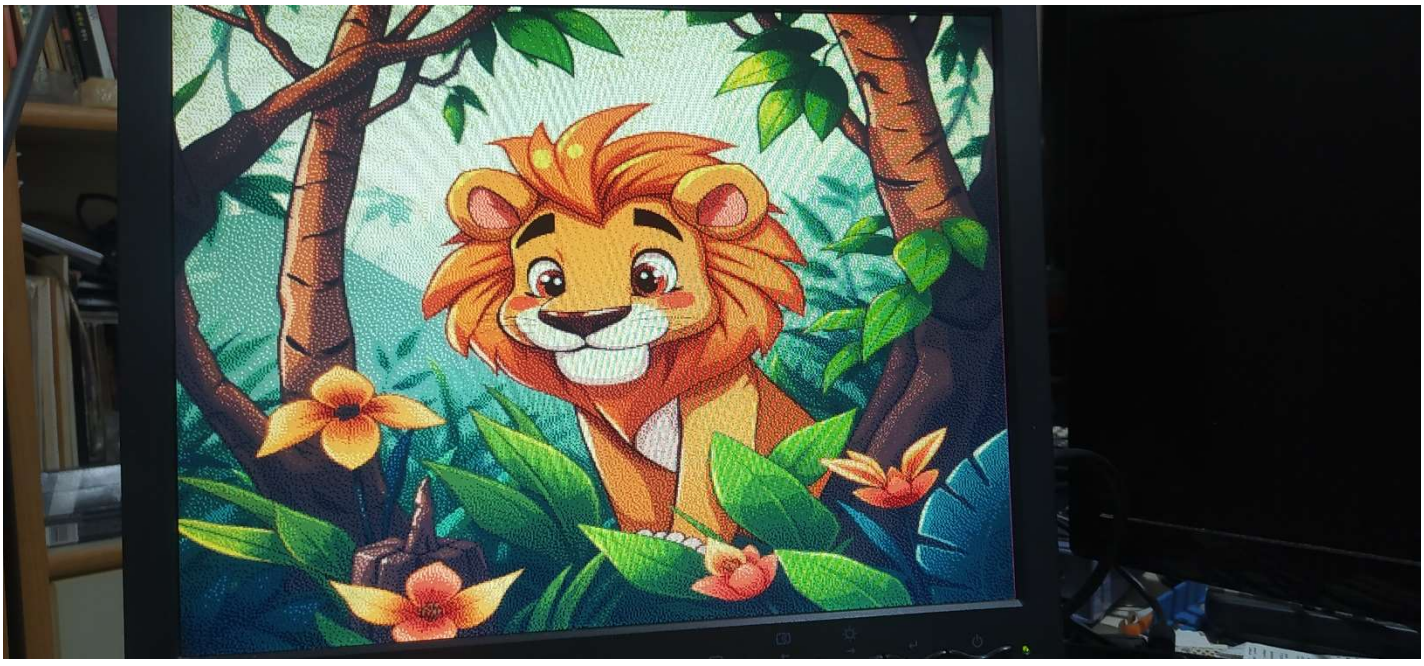
In mode 144, the standard mode-8 FLASH bit is replaced by an intensity bit and as a result 8 lighter colours are available. In this mode you can use the set of the following specialized QL Core commands (available with lrespr fu_bin) : **FCON, FCOFF, FINK and FPAPER**. **These commands are not compatible with the pointer environment so don't use them before or after loading it.**

At mode 32 there are 2 commands **CLS256** and **PCH256** that can be used to paint the screen and print a character in any position with any of the 256 colors. Also FPLOT x,y,colour sets the colour of the pixel @x,y.

Mode 48, 256x256 4096 colours

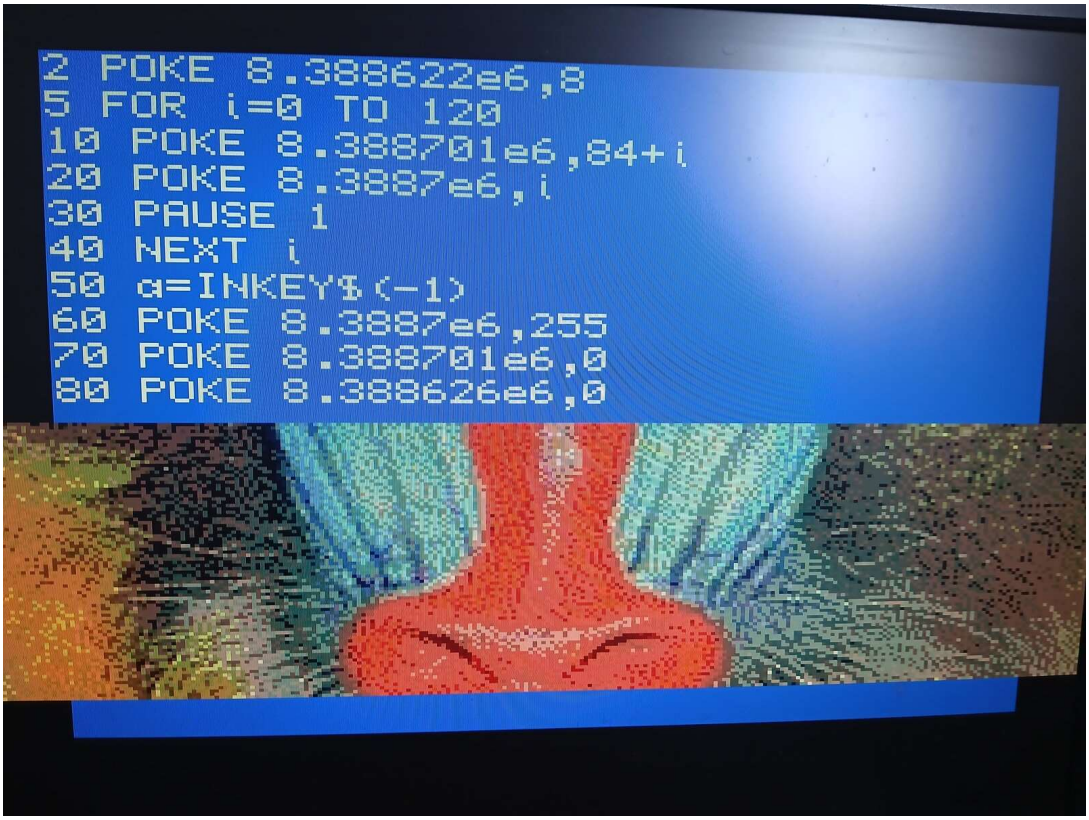


Mode 37, 512x384 256 colours



Resolution Mix

There is a way to mix other resolutions with the 64K (256x256x256, 512x256x16) video resolutions, this is currently not fully tested.
This is achieved by writing the scan line number (0 to 255) you want to start the 256x256x256 resolution at 8388700 address and writing the scan line number you want this resolution to end at 8388701 address.
To see the screen mixing enter: lrun demos_testvm preferably after you have run the test256 program.



Sprites

The sprite engine addresses three sprite banks each with 4K dedicated ram that each can store 30 16x16 pixel hardware sprites. Firmware up to version 3.1 supports 50 sprites; newer firmware supports 90.

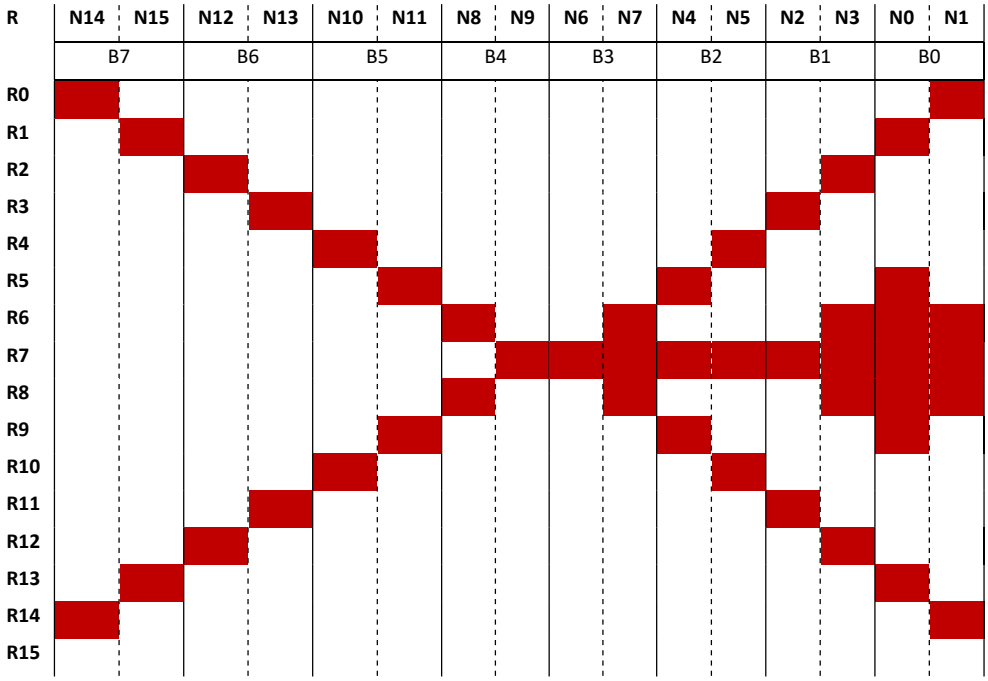
Sprites have 15 colors + transparency and work in a 256 x 256 addressable plane.
The first 128 bytes in every 4K block hold the parameter table of the sprites; 4 bytes for each sprite.

The parameter table is divided in 30 records, with a 4 byte record per sprite as follows:

Byte 0: Enable, if equal to 1 sprite is enabled (enabled means visible or not)
Byte 1: X position, 0..255
Byte 2: Y position, 0..255
Byte 3: Reserved (possibly for magnify/mirror/rotate)

Pattern (graphic) data begins at offset 128 in each bank. Every sprite pattern is 128 bytes long and they are sequentially stored.

Each byte in sprite graphic data contains color for 2 pixels (16 nibbles 4 bits each). Each sprite is made of 8 bytes (representing 16 pixels on the x axis) times 16 (representing each line on the y-axis). Hi nibble is for the right pixel, low nibble is for the left pixel.



Sprites pixels can be transparent by assigning colour $F in its pattern, this is the transparency colour. Each nibble is a palette index (0–14); $F is transparent. The default palette gives the IRGB colours (I=intensity), see Sprite Palette below.

Although there are 3 banks, the 90 sprites can be easily addressed by number in a linear way using the next formula
Sprite N parameter address = $C00000 + (N div 30)*4096 + (N mod 30)*4
Sprite N data address = $C00000+128+ (N div 30)*4096 + (N mod 30)*128

Set parameters example in C

```
static void set_sprite( int s, int en, int x, int y)
      {
       volatile unsigned char *addr;
       int bank=s/30; int ss=s%30;
       addr=(volatile unsigned char *)(0xC00000+bank*4096+ss*4);
       *addr=en;
       addr++;  *addr=x;
       addr++;  *addr=y;
```

```
}


static void set_sprite_data( int s, unsigned char *d, int frame, int colOR)
{
 volatile unsigned char *addr;
int bank=s/30; int ss=s%30; int j;
addr=(volatile unsigned char *)(0xC00000+bank*4096+128+ss*128);
      for (j=0; j<128; j++)
          {
           *addr=(d[j+frame*128] | colOR);
           addr++;
          }
}
```

The lower number sprites are on top when they overlap with others.
Two new sprite relevant commands are **SETSPR (**or **SETSP** in older fu_bin version**)** and **CLRSPR (CLRSP).**

Sprite Palette

There is a 1440 byte long memory area at address 12648448 that is divided in 90 blocks of 16 bytes, one block for each sprite. Blocks are serially arranged in the sprite ascending order.
In each block the first byte contains the 8bit value of the colour that corresponds to the sprite's colour 0 , the second byte contains the 8 bit value that corresponds to colour 1 ... and so on up to colour 14 (15 has no use it is the transparency value)
So to set colour 5 of sprite 23 you have to write the 8bit colour value (1 byte) to address 12648448+23*16+5 (POKE 12648448+23*16+5, colour)

The pattern of the 256 colour to choose from are in the RRRGGGBB form. By default the palette memory contents implement the pattern IRGB for the 16 colours of each sprite.

Example: random values for galaxp game sprites

```
10 FOR I=0 TO 799
20 POKE 12648448+I, RND(255)
30 NEXT I
40 EW DEMOS_GALAXP
```



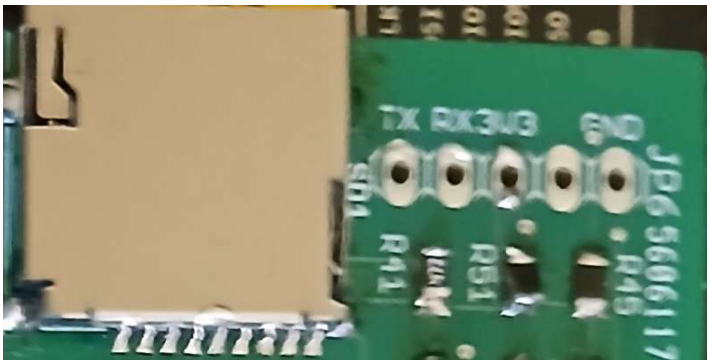
YM/AY Audio

An **YM2149F** (**AY-3-8910-A** compatible) PSG is implemented in the FPGA to provide QSound-like facilities to QLion. There is the **Sound** basic command that is compatible with the Qsound command to produce basic sounds from the 3 channels.
You can also use the AYSET command and the AYGET function that access the ic registers to produce different sounds and noise (read the IC's datasheet it explains).
The YM2149 ic in QLion runs at 2.5Mhz.
There is a very simple player called YMPLAY that supports playback of ym files of version 5 and 6, interlaced or not, uncompressed.
Usage ew ymplay;"filename"
Example ew ymplay;"demos_test_ym"

PCM Audio

The use of PCM audio requires to fill the buffer with the digital audio data, set the playback frequency and start the playback. When the end of the buffer is reached the playback continues again from the start. By filling the first half of the buffer while playing the second half a bigger music file can be played.
The buffer starts at $C04000 and has 8Kbytes size. To start or stop the playback a 1 or 0 must be written at $800034.
The playback speed is set by writing a word at $800036 containing the 5000000/(desired frequency) value.
To be able to tell where the playback index is currently you can read the word at $800034.
Data in the buffer is the form of unsigned integers, sample data is 8bit and every word of the buffer contain the left and the right channel at the high and low byte. LRLRLRLRLRLRLR…
I use audacity program to convert audio files to raw data.
There is a simple player called DPLAY, usage EW dplay;"filename frequency"
Example ew dplay;"demos_pol_raw 8000"

Serial/ESP



In the photo you see the serial port pinout: Transmit, Receive, 3.3V, NC, GND
In latest pcb the NC pin is omitted so the pinout is: Transmit, Receive, 3.3V, GND
It has 512bytes/512bytes Receive/Transmit hardware buffer.

The fast serial port signals are 3.3V and can be connected using a common ttl to usb board to a pc or a ESP8266 board , the device is enabled by loading a driver with **lrespr fserial_bin**.
Optionally a ESP12E board can be soldered under the pcb, the communication with the ESP is done using the ESP1 or SER3 device. You can open it and write AT commands to the ESP and read the results.
In basic use the BPUT #channel,ascii_code command instead of the print #channel, string command.
When ESP12E is soldered to the pcb the fast serial port is dedicated to the esp connection.

Example sending/receiving byte (d1) to ser3

```
ESP_INOUT EQU       $800025
ESP_STAT  EQU       $800024
ESP_RD    EQU       0
ESP_DR    EQU       1
ESP_R     EQU       1
ESP_W     EQU       2

read   moveq   #1,d0
       btst.b  #ESP_DR,ESP_STAT ; is there input?
       beq.s   rexit            ; no, return no input
       move.b  ESP_INOUT,d1
       move.b  #ESP_R,ESP_STAT  ; remove read value from buffer
       nop
       move.b  #0,ESP_STAT
       moveq   #0,d0            ; return OK, data in d1
rexit  rts

send
       moveq   #1,d0
       btst.b  #ESP_RD,ESP_STAT ; is there room in the buffer
       beq.s   rexit            ; no return with d0<>0
       move.b  d1,ESP_INOUT
       move.b  #ESP_W,ESP_STAT  ; write to output buffer to be send
       nop
       move.b  #0,ESP_STAT
       moveq   #0,d0            ; return OK
       rts
```

Real Time Clock

You can set the QL time at boot using the command (exactly like this):
SDATE RTCGET(6)+2000,RTCGET(5),RTCGET(4),RTCGET(2),RTCGET(1),RTCGET(0)
This command reads the time from the rtc ic and sets the ql time.
To set the time in RTC ic you can enter **run setrtc**, this basic program use the RTCSET added sbasic command to set the correct values to RTC ic registers.

Storage QLSD

A SPI interface is implemented in the FPGA to access a SD-Card and the QLSD driver from SMSQ has been converted to use this interface.
It works with a FAT32 formatted HCSD Card in which **.win** image files should be written. The files must not be fragmented and they should be the first files copied to the card.
The images in the SD-CARD with names QXL.WIN, QXL2.WIN … are initially mapped to the WIN1_, WIN2_ … devices, up to 8 images.
The standard QLSD commands WIN_USE, WIN_DRIVE, WIN_CHECK are added to SuperBASIC.
A new command is added **ISD** that allows to swap the SD card, one must be sure the card is not accessed and remove it, insert another card and enter the ISD command, the new SD card will be recognized and the win images on that card mounted.

Fat support utilities

SDIR, SCOPY, QCOPY

For a listing of the SD card directory enter: ew sdir,#1 (and use space to scroll)
To copy a file from the SD card into a win image: ew scopy;'filename_fat filename_ql'
To copy a file from the image to the fat in sd: ew qcopy;'filename_ql filename_fat'
Be careful those commands overwrite without asking for confirmation (not implemented yet)

SuperBASIC Extensions

Some commands and functions added to super basic by loading the fu_bin with: lrespr fu_bin

FSLOW n
It slows down the execution speed for any game or application that is not usable at the higher speed. The parameter n takes values from 0 to 63, 0 is normal speed and 63 is the slowest speed.

FCON
When in mode 8 this enables the 16 colour by using the flash bit as intensity.

FCOFF
This command disables the 16 colour mode.

FPAPER #ch,n
When in 16 colour mode this command sets the paper color of a window channel, n=0 to 15
This command doesn't work with pointer environment!

FINK #ch,n
When in 16 colour mode this command sets the ink color of a window channel, n=0 to 15
This command doesn't work with pointer environment!

FPLOT x,y,col
When in 16 colour mode this command sets the colour of the pixel at x,y to col.
x=0..255 y=0..255 col=0..15, (0,0) is the top left corner.

ISD Used to hot swap sd cards, explained in the "Storage QLSD" paragraph.

SETSPR no,enable,x,y
Sets a sprite visibility and position, was SETSP in older fu_bin version
no=sprite number 0 to 89
enable=0 hidden or 1=visible
x=0 to 255, y=0 to 255, (0,0) is the top left corner.

CLRSPR
Hides all the sprites from the screen, was CLRSP in older fu_bin version

PCH256 x,y,code,col
Prints a character at x,y when at 256x256 with 256 colours mode 32.
Code = character's ascii code
Col = colour, 0 to 255
x,y take values from 0 to 255. (0,0) is the top left corner.

CLS256 col
Paints the whole screen in colour **col** when in mode 32.

SOUND ch,freq,vol
Sound plays a tone until another sound or a nosound command is executed.
ch = sound channel, 0 to 2
freq = the frequency
vol = volume, 0 to 15
When given with only the channel parameter it mutes the specified channel.

NOSOUND
It mutes all the sound channels.

RTCGET(n)
This is a function that returns the value of the nth register of the RTC ic (adjusted as needed).
Print RTCGET(6) prints the years since 2000,
5 returns the month, 4=day, 2=hour, 1=minute, 0=second

RTCSET n,v
This command writes to the RTC nth register the value v

AYSET n,v
Writes at the nth register the v value, see AY-3-8910 datasheet
n 0..14, v 0..255

AYGET(n)
This is a function that returns the value of the nth register of the sound ic

FBAUD n
Sets SER3 baud rate to 10MHz/(n+1), so FBAUD 42 sets baud rate to 232558. Default 115200.

FTIMER()
Function that returns the lower 16bits of User timer

FSETTIM n
Sets the user timer frequency to 10MHz/(n+1)

New commands added 8/2026. These commands work in all modes

FCLS col Fills screen with colour col.

FMODE [modenum] Set video mode *modenum*, when used with no argument it sets standard QL mode 4

FCHAR x1,y1,char,col Prints a character of ascii value *char* of colour *col* at x1,y1
FPLOT x,y,col Does what you guessed
FLINE x1,y1,x2,y2,col Does what you guessed
FCIRCLE x1,y1,r1,col Does what you guessed
FRFILL x1,y1,x2,y2,col Fills a rectangular area defined by x1,y1 upper left corner, x2,y2 lower right corner, with colour col.

FRCOLOUR (x,y) Returns the value of pixel's x,y colour

Memory Map

HEX	DEC	Description
$000000	0	48K SYSTEM (patched QDOS/MINERVA) ROM + 16K QLSD ROM
$010000	65536	32K that can be used for driver ROM
$018000	98304	QL I/O
$020000	131072	32K VRAM + 32K RAM (SuperBASIC variables) / Minerva 2nd screen VRAM
$030000	196608	3648K RAM ($030000–$3BFFFF)
$3C0000	3932160	Not used (gap between system RAM and video RAM)
$4C0000	4980736	256K Video RAM
$800000	8388608	256 addresses for QLion I/O
$C00000	12582912	Sprite RAM bank 1, 4K
$C01000	12587008	Sprite RAM bank 2, 4K
$C02000	12591104	Sprite RAM bank 3, 4K
$C04000	12599296	Digital Audio buffer 8K
$C08000	12615680	16K special ram for time critical parts of programs
$C10000	12648448	1440 bytes Sprite Palette

Memory Mapped I/O Registers

Hex Addr	Dec Addr	Size	R/W	Description
QLion ID				
$80001E	8388638	WORD	R	ID=$4C30 “L0”
$800048	8388680	WORD	R	Core Version. High byte: Major Rev. Low byte: Minor Rev.
Lion-SPI (QLSD)				
$800001	8388609	BYTE	W	bit0=1 (pulse) send/receive a byte
$800003	8388611	BYTE	W	L-SPI clock divider
$800005	8388613	BYTE	W	byte to send
$800007	8388615	BYTE	R	received byte
$800009	8388617	BYTE	R	bit0 = interface ready
$80000B	8388619	BYTE	W	Set CS = bit0
SPEED CONTROL				
$800017	8388631	BYTE	W	Bit0 = 1 Slow down cpu
$800013	8388627	BYTE	W	amount to slow down cpu in bit5 to bit0, 0 to 63
REAL TIME CLOCK I2C interface				
$800021	8388641	BYTE	R/W	W: I2C data out, R: data in
$800023	8388643	BYTE	W	W: bit0=I2C enable (1) or I2C disable (0) R: bit0=busy, bit1=error
VIDEO MODES				
$80000E	8388622	BYTE	R/W	***Write:*** By writing the mode number in this address that mode is enabled. Read: Reads the contents of the register (mode number)
$800010	8388624	WORD	R	Bit 15 = Vertical Sync (ON/OFF) (0/1) Bit 14 = Horizontal Sync (ON/OFF) (0/1) Bits 0…9 = Current Scan Line (0 to 805)
$800012	8388626	BYTE	W	bit0=1 enables 16 colors for mode 8
ESP/UART (SER3)				
$800025	8388645	BYTE	R/W	W: SER/ESP data out, R: data in
$800024	8388644	BYTE	R/W	W: bit0 = ser/esp_Write, bit1 = ser/esp_Read. R: bit1 = busy, bit0 = data_ready
$80002C	8388652	WORD	W	Baud rate frequency divider, Baud=10Mhz/(divider+1), Example: 86 gives about 115200

Hex Addr	Dec Addr	Size	R/W	Description
AY/YM AUDIO				
$800028	8388648	BYTE	R/W	AY data read/write
$80002A	8388650	BYTE	W	AY control register
DIGITAL AUDIO				
$800034	8388660	WORD	R/W	R: returns the current pointer in the buffer during playback. W: bit0 = start playing trigger
$800036	8388662	WORD	W	Period in 1/5Mhz steps, example 625 = 8Khz rate (5000000/8000=625)
COUNTERS				
$800030	8388656	LONG	R	Counter 1Mhz by default, 32bit long
$80003C	8388668	WORD	W	Frequency divider, the above Counter divider gives a 10Mhz/(divider+1) frequency, default value 9 (10MHz ÷ 10 = 1MHz).
$800044	8388676	LONG	R	21.6Mhz fixed 32bit long
$800064	8388708	LONG	R	10Mhz fixed counter (used for Microdrive access), 32bit long
VARIOUS SETTINGS				
$800060	8388704	BYTE	W	Bit 0: Write 1 to make ROM writeable (Useful for ROM patching) Bit 1: set = Enable AURORA video colour patterns Bit 2: set = Enable mouse (enable always after loading ptr environment)
$80003E	8388670	BYTE	W	Write 3 to switch to 68020 and 0 to return to 68K
$80005C	8388700	BYTE	W	Resolution mix: scan line (0–255) where the 256x256x256 area starts
$80005D	8388701	BYTE	W	Resolution mix: scan line where the 256x256x256 area ends

Copyrights

The TG68K.C core used is a copyright of Tobias Gubener under the terms of the GNU Lesser General Public License as published by the Free Software Foundation.

YM2149/AY-3-8910 module is a «Copyright (c) of MikeJ - Jan 2005»

The SD card driver is copyright Tony Tebby, Wolfgang Lenerz and Marcel Kilgus modified to use QLion hardware by Theodoulos Liontakis. It is free software under the terms of the SMSQ/E license and the source code is part of the SMSQ/E distribution.

LICENCE FOR SMSQ/E :

Copyright (c) 1989-2012, by

- Tony Tebby
- Marcel Kilgus
- Bruno Coativy
- Fabrizio Diversi
- Phoebus Dokos
- Thierry Godefroy
- Jérôme Grimbert
- George Gwilt
- John Hall
- Mark Swift
- Per Witte
- Wolfgang Lenerz

collectively called the "COPYRIGHT HOLDERS".

All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL COPYRIGHT HOLDERS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

ND-QLi driver by Martyn Hill

QLion Gold hardware design, VHDL logic code, utilities and basic extensions are copyright of Theodoulos Liontakis 2025.

This is a hobby project, no responsibility is accepted for the loss of data or consequent damage of any kind resulting from the use of this design, hardware and/or software.